NEMOX AI | RICO-7

THREE COMPLETE FIELD-BASED DEVELOPER TOOL SOLUTIONS | Architecture v1.1 | 1 November 2026

Independent engineering reference based on the 17-page field-based architecture. Publicly described nemox capabilities are contextual inputs, not inspected private systems.

Executive architecture

One field vocabulary supports three independently executable inspection tools: A reconstructs request interpretation, B captures corrections and later retrieval, C evaluates dependency completeness. The prototype implementations are separate FastAPI and React/Vite applications with a shared typed field library.

Implementation scope: functional local prototypes, synthetic logistics data, SQLite persistence, deterministic processing and API smoke tests. Production event sourcing, access controls, genuine model integration, operational adapters and complete UI coverage remain engineering work.

I. Challenge and domain reconstruction

User: Operator or customer submitting an operational request.

User intent: Requested outcome inferred from the original message and referenced context.

Requirement: Typed constraint on a logistics action, retaining its source span.

Operational context: Shipment, customer, carrier, rate, document and workflow facts relevant to a request.

Decision: Candidate action and immutable context snapshot at a particular point in time.

Feedback: Explicit user edit or approval tied to an earlier decision.

State transition: Versioned change caused by a source event.

Longitudinal understanding: Confirmed, scoped relations retrieved across interactions.

II. Twelve field contracts

F01 User: user_id, role, organization_id, interactions; owner identity adapter; user/tenant scope. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F02 Request: request_id, text, channel, occurred_at, source_ref; owner ingestion; initial to enriched. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F03 Operational Context: customer_id, shipment_id, workflow_state, documents, constraints; owner context resolver. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F04 Intent: objective, entities, temporal_delta, unresolved_refs; owner extractor; derived. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers.

Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F05 Requirement: requirement_id, attribute, value, unit, evidence_ref, status; owner requirement mapper. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F06 Agent Decision: decision_id, request_id, action, snapshot, model_version; owner decision collector. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F07 Feedback: feedback_id, decision_id, correction, explicit_reason, status; owner feedback processor. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F08 Relation: relation_id, subject, predicate, object, source_ref, validity, version; owner relation service. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F09 State: field_id, previous_state, current_state, cause_event, version; owner transition manager. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F10 Understanding Gap: gap_id, required_attribute, blockers, question, resolution_state; owner gap engine. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F11 Developer Inspection: trace_id, source refs, relation graph, decision history, gaps; owner projection API. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

F12 Longitudinal Context: customer_id, confirmed corrections, valid_from, superseded_at; owner retrieval service. Data validation: nonempty identifiers, source references for assertions, monotonic version and scoped consumers. Persistence: SQLite records and immutable event rows in prototype; migration to relational version tables for production.

III. Seven-pass reconstruction, separately applied

A Intent Inspector

Pass 1, Initial domain reconstruction: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 2, Dependency reconstruction: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation

ownership.

Pass 3, Boundary reconstruction: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 4, State reconstruction: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 5, Feedback reconstruction: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 6, Inverse validation: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 7, Architecture consolidation: request to historical shipment resolution and evidence-linked requirements. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

R1 domain: freight operator; R2 information: original request and context; R3 semantic: typed attributes and relations; R4 decision: candidate action and prerequisites; R5 feedback: corrections and confirmations; R6 state: versioned lifecycle; R7 longitudinal: scoped historical retrieval. Inverse sweep: developer view back to API, service, field projection, persisted event and original request.

B Feedback Engine

Pass 1, Initial domain reconstruction: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 2, Dependency reconstruction: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 3, Boundary reconstruction: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 4, State reconstruction: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 5, Feedback reconstruction: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 6, Inverse validation: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 7, Architecture consolidation: decision snapshot to correction, confirmation and later retrieval. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

R1 domain: freight operator; R2 information: original request and context; R3 semantic: typed attributes and relations; R4 decision: candidate action and prerequisites; R5 feedback: corrections and confirmations; R6 state: versioned lifecycle; R7 longitudinal: scoped historical retrieval. Inverse sweep: developer view back to API, service, field projection, persisted event and original request.

C Gap Detector

Pass 1, Initial domain reconstruction: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 2, Dependency reconstruction: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 3, Boundary reconstruction: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 4, State reconstruction: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 5, Feedback reconstruction: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 6, Inverse validation: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

Pass 7, Architecture consolidation: workflow target to prerequisite closure, clarification and re-evaluation. Output: dependency edge inventory and explicit producer/consumer boundary. Change from preceding pass: progressively add data contracts, transition guards, feedback links and implementation ownership.

R1 domain: freight operator; R2 information: original request and context; R3 semantic: typed attributes and relations; R4 decision: candidate action and prerequisites; R5 feedback: corrections and confirmations; R6 state: versioned lifecycle; R7 longitudinal: scoped historical retrieval. Inverse sweep: developer view back to API, service, field projection, persisted event and original request.

IV. Solution A: User Intent & Context Inspector

Business rule: a historical reference cannot become a concrete shipment without a unique context match. Relative dates retain a delta until baseline delivery is resolved. A source segment is mandatory for every derived requirement.

Processing: POST /api/analyze -> parse request -> extract reference and relative date -> query customer-scoped shipment history -> construct F02/F04/F05 -> unresolved markers -> persist trace -> inspect GET /api/traces/{id}.

Failure: multiple historical shipments remain ambiguous; missing historical data remains unresolved; input validation returns HTTP 422. Demo: same shipment as last week, two days earlier, yields delta -2 and missing baseline when no shipment exists.

```
POST /api/analyze {"request":"Same shipment as last week, but two days earlier."}
GET /api/traces/{trace_id}
POST /api/history {"shipment_id":"S-1","delivery_date":"2026-11-08","carrier":"C-A"}
```

V. Solution B: Recursive Feedback & Decision Trace Engine

Business rule: correction and confirmed reusable preference are different facts. Decision snapshots remain immutable; review creates a separate record. Context retrieval uses only confirmed corrections scoped to the customer.

Processing: POST /api/decisions -> decision snapshot; POST /api/feedback -> proposed correction; POST /api/confirm -> confirmed or rejected review; POST /api/context/retrieve -> retrieval manifest; POST /api/compare -> deterministic comparison.

State model: proposed -> confirmed or rejected. Future implementation should version relation states and validate optimistic concurrency before writes. The prototype records events and reviews without a production transactional outbox.

```
POST /api/decisions {"request":"Quote freight","carrier":"Carrier A"}
POST /api/feedback {"decision_id":"<id>","corrected_carrier":"Carrier B"}
POST /api/confirm {"correction_id":"<id>","confirmed":true}
POST /api/compare {"request":"Quote freight again"}
```

VI. Solution C: User Understanding Gap Detector

Business rule: choose_cheaper_carrier requires baseline_shipment, rate_options and delivery_limit. Baseline requires shipment_reference; rates require origin and destination. A clarification resolves a missing leaf prerequisite; the dependency graph is re-evaluated after each answer.

Processing: POST /api/gaps/analyze -> build prerequisite graph -> recursively evaluate required leaves -> emit clarifications -> POST /api/gaps/resolve -> new analysis and remaining missing fields. Cycle detection raises an error in graph evaluation.

Algorithm: recursively traverse dependency edges, treating available facts as satisfied, missing leaves as questions and alternative prerequisite groups as candidate sets. Choose a minimum-cardinality candidate set for clarification. Production extension: cost-weighted question selection and dynamic workflow templates.

```
POST /api/gaps/analyze {"request":"Ship this as usual, but use the cheaper option"}
POST /api/gaps/resolve {"analysis_id":"<id>","attribute":"shipment_reference","value":"S-1"}
```

VII. Shared implementation and coupling

Shared library fieldcore.py defines SourceRef, FieldRecord, Relation, Event and a transition guard. store.py implements SQLite record/event storage. Each solution has its own backend/main.py, requirements, Dockerfile, frontend/src.tsx, Vite configuration and pytest test. No tool depends on another tool at runtime.

Couplings: F02 -> F04 by extraction; F03 -> F04 by reference resolution; F04 -> F05 by requirement mapping; F06 -> F07 by decision_id; F07 -> F08 by confirmed correction; F08 -> F12 by scoped retrieval; F05 -> F10 by dependency evaluation; all projections -> F11 by trace references.

API errors: 422 invalid body, 404 unknown record, 409 duplicate review. Local prototype does not implement tenant authorization or idempotency keys; those are required before real integration.

VIII. Two-developer allocation

Developer A owns Python domain logic, API contracts, SQLite storage, tests and Docker. Developer B owns three React/Vite inspection shells, typed API interactions, relation graph rendering and demo. Interface handoff: JSON request/response bodies in OpenAPI at ports 8011, 8012, 8013.

IX. Six-hour hackathon plan

Hour 1: freeze field IDs, request/response contracts and synthetic fixtures.

Hour 2: establish independent backend APIs and SQLite storage; frontend shells in parallel.

Hour 3: implement A extraction and B decision/feedback lifecycle; UI trace view.

Hour 4: implement C dependency traversal and B retrieval; UI graph view.

Hour 5: run API tests, frontend build and three demonstration scenarios.

Hour 6: correct defects, replay demos, prepare engineering pitch and record open production dependencies.

X. Architecture validation and remaining dependencies

Executed API tests: one passing test per solution, 3 total. Frontend compilation, Docker builds and Playwright E2E tests were not executed. The implementations are demonstrable prototypes rather than production-complete systems.

Production dependencies: tenant authentication, encryption and retention policy, transactional event outbox, idempotency and replay, relation version history, real LLM adapters, nemox-specific APIs, TMS connectors, approval semantics, semantic extraction for varied RFQs, richer frontend workflows and complete end-to-end tests.

XI. Product improvement mapping

Quotation: connect RFQ requirement trace to historical quote edits; metric: unresolved references and repeated corrections.

Export: evaluate document prerequisites before AWB generation; metric: missing document fields before approval.

Import: trace pre-alert references and missing documents; metric: unresolved shipment references.

Accounting: correlate invoice mismatch decisions with reviewer corrections; metric: correction recurrence and decision trace coverage.

Cross-workflow: propose Customer -> Quote -> Booking -> Shipment -> Invoice -> Feedback link registry with transfer manifests; metric: context continuity across workflow boundaries.

Appendix: executable project map

.github/workflows/tests.yml

01_user_intent_inspector/Dockerfile

01_user_intent_inspector/README.md

01_user_intent_inspector/backend/__init__.py

01_user_intent_inspector/backend/main.py

01_user_intent_inspector/frontend/index.html

01_user_intent_inspector/frontend/package.json

01_user_intent_inspector/frontend/src.tsx

01_user_intent_inspector/frontend/tsconfig.json

01_user_intent_inspector/frontend/vite.config.ts

01_user_intent_inspector/requirements.txt

01_user_intent_inspector/tests/test_api.py

02_recursive_feedback_engine/Dockerfile

02_recursive_feedback_engine/README.md

02_recursive_feedback_engine/backend/__init__.py

02_recursive_feedback_engine/backend/main.py

02_recursive_feedback_engine/frontend/index.html

02_recursive_feedback_engine/frontend/package.json

02_recursive_feedback_engine/frontend/src.tsx

02_recursive_feedback_engine/frontend/tsconfig.json

02_recursive_feedback_engine/frontend/vite.config.ts

02_recursive_feedback_engine/requirements.txt

02_recursive_feedback_engine/tests/test_api.py

03_understanding_gap_detector/Dockerfile

03_understanding_gap_detector/README.md

03_understanding_gap_detector/backend/__init__.py

03_understanding_gap_detector/backend/main.py

03_understanding_gap_detector/frontend/index.html

03_understanding_gap_detector/frontend/package.json

03_understanding_gap_detector/frontend/src.tsx

03_understanding_gap_detector/frontend/tsconfig.json

03_understanding_gap_detector/frontend/vite.config.ts

03_understanding_gap_detector/requirements.txt

03_understanding_gap_detector/tests/test_api.py

README.md

docker-compose.yml

shared/__init__.py

shared/fieldcore.py

shared/store.py

Source and provenance

Primary architecture: NEMOX_AI_RICO7_FIELD_BASED_DEVELOPER_TOOL_ARCHITECTURE.pdf (17 pages, 1 November 2026), provided by user. Public nemox capabilities are cited in that source as company claims. All new components and code are independent proposed reference implementations.